DS3103 Webutvikling – Exam Rapport Autumn 2024

2. Overview of functionality you have made in frontend, and short about the techniques used in context of the functionality.

- **AddStaff (**Brukeren kan lage og legge til ansatte.)
    - useState () – håndterer lokale tilstander som newStaff, image, preview og status som holder øye med brukerinput, valgt bilde, forhåndsvisning av bilde, og status for bildeopplastningen.
    - useContext () – henter tilgang til funksjonen addStaff fra StaffContext.
    - handleUserInput oppdaterer tilstanden til newStaff dynamisk når input-felten fylles ut.
    - Opplastning av bilder med forhåndsvisning. Her brukte jeg URL.createObjectURL() metoden. Slik at man sikrer seg rett fil før opplastning.
    - Knapp for å legge til brukeren når alle feltene er fylt ut slik at den blir lagret i databasen og kommer opp på «hovedsiden» (Staff Page).
    - Responsivt design laget med Tailwind CSS slik at løsningen er brukbar på enheter med forskjellige størrelser.

- **DeleteStaff (**Slette en ansatt)
    - useContext () – henter tilgang til funksjonen onDeleteStaff() fra StaffContext.
    - Knapp med symbolet til en søppelbøtte for å slette den ansatte.
    - Sletter den ansatte fra backend slik at de også blir borte fra frontend.
    - Får en alert når ansatte har blitt slettet.
    - Knyttet opp mot id slik at den rette brukere blir slettet.

- **StaffItem** (Ansattdata)
    - Designet med Tailwind CSS for å fremvise en ansatt, og deres informasjon.
    - Hvis den ansatte en da jobber eller ei. Rød farge hvis de ikke jobber, og grønn hvis de jobber basert på statusen.

- **StaffList** (Liste over de ansatte)

  - Grid-layout ved bruk av Tailwind for håndtering av responsivt design.
  - getStaffItems () går igjennom staffList og generer en <li> for hver ansatt og disse inneholder en StaffItem som viser data for en individuell ansatt.
  - Hvert staffItem får ne unik nøkkel basert på indeksen i i listen slik at man får riktige oppdateringer i DOM.
  - Array.map brukes for å iterer over staffList og lage komponenter for hver ansatt.

- **UpdateStaff** (Oppdatere/redigere en ansatt)
  - Popup-vindu via en knapp for å kunne oppdatere en ansatt.
    - Mulighet for å lukke med en «lukk»-knapp eller avbryte handling (cancel).
  - loadStaffData (): henter data for en ansatt via staffId.
  - handleInputChange (): Input-feltene for å kunne oppdatere staffData.
  - handleImageUpload (): Mulighet for bilde-opplastning.
  - handleUpdateStaff (): Lagrer dataen ved å klikke på save-knappen.

3. Write about what you have done in your frontend solution, code and design, to follow Universal Design (Universell Utforming).

For å følge UU har jeg prøvd å holde koden tydelig strukturert og semantisk for at den skal være lettere å tolke. Jeg har også implementert tabindex, og har fokusindikatorer på interaktive elementer som inputfelt og knapper.
Jeg har holdt en klart kontrast mellom tekst og bakgrunn, og lagt på tydelige hover effekter der det er fargeendringer. Det er brukt responsivt design slik at der skal være mulig å bruke løsningen på forskjellige skjermstørrelser.
Jeg har fokusert på en lett forståelig og tydelig nettside der formålet er at brukeren ikke skal trenge å tenke så mye når de bruker den. Funksjoner gir mening fordi de har vært borti det før, og er tydelig markert med enten symbol-knapper eller tekst-knapper.

Kommentar/kilde til koden:

Da jeg startet oppgaven var det ikke tydelig beskjed med tanken på hvor mange databaser vi trengte å lage, og derfor har jeg navngitt tabellen TrumpStaff istedenfor bare staff. Siden jeg lagde alle tre databasene helt til å starte med, og brukte mye tid på backend så fikk jeg lite tid til å forandre på det senere.

Ved besvarelse av denne eksamenen har jeg brukt ulike kilder for å gjennomføre arbeidet mitt. Dette inkluderer materialer fra forelesningene(hovedsaklig), nettsøk via Google og andre nettsteder relatert til programmering. I tillegg har jeg benyttet ChatGPT som et verktøy for feilsøking og for å få ideer til hvordan oppgavene kunne ha blitt løst.